Name : Muhammad Mahad Khan

Student ID : 35718625

Section A: Describing how the solution successfully meets all requirements of the scenario given

After taking a good look, examining the assignment specification and the Java program provided for this task, I reached the conclusion that the program aligns fairly well with the scenario's description. The program is designed to manage basic user registrations, enabling users to create, log in, and manage their own accounts with various options, such as changing passwords, emails, or deleting their accounts completely.

The way the program starts is pretty straightforward. When you launch it, you immediately see a clear main menu with three options (registering, logging in, or quitting). This part is accurate because it exactly follows the assignment specifications/instructions. The program stores users in an ArrayList, this means it can easily handle lots of users at once. Each user is represented by an object from a class called RegisteredUser, which has three fields a unique ID number, an email address, and a password.

The registration option does exactly what you'd expect which is to ask user for their email address and password to register their UserID. The program prompts the user for an email address and password, and it checks that these details are valid or not. The email needs to include an "@" symbol, and this symbol cannot be the first or last character. This is a very good example of validation because it prevents obvious mistakes like "email@" or "@email". For the password, the program checks three important things, it has to be at least eight characters long, have both uppercase and lowercase letters, and at least one digit. These checks are practical and align with the assignment scenario’s requirements.

Once the validation process passes, a new RegisteredUser is created with a unique user ID number. The ID numbers start from 1 and increments for each new registration which meets the requirement that each user should have a unique ID. After registration, the program clearly confirms success and ouputs all user details right away, which is user-friendly and helpful.

For the loggging in option, users can enter their email and password. The program then checks through the list of registered users to find a matching email and password pair. If it finds a match, users get logged in successfully and enter another menu. The program then asks the user for another try if the email or password is incorrect. There should only be 3 login attempts according to the assignment specifications which is where the program lacks because there are unlimited attempts in the program provided. It is something that's easy to fix, a simple loop and a counter would sort it out.

Inside the logged in user menu, users can easily print their details, change their email or password using straightforward methods, or delete their account. If they delete their account, the program removes them from the ArrayList, and the user is sent back to the main menu. Logging out is also a very straightforward process.

Overall, besides the unlimited login attempts problem, the program pretty much follows all the requirements given in the scenario in the assignment. The program is easy to understand and follows logical steps, however there are no comments which would make the code more readable.

Section B: Describing the Approach to Object-Oriented Design

Taking a close look at the Java program provided, it is clear that it follows straightforward approach to Object-Oriented Programming (OOP). Object Oriented Programming is all about organizing code into objects, each object is responsible for a specific set of tasks, this is done to make the program easier to understand, manage and update. This program provided mainly uses two classes, RegisteredUser and Main.

The RegisteredUser class has a pretty simple role which is to store user information. The program uses three main fields for each user, a unique UserID, an email, and a password. These fields are private, which is very important because it keeps the data protected, following the principle of encapsulation. To read and modify these fields, the application offers public getter and setter methods. This is normal practice and ensures that you have control over how and when the data is modified.

Another interesting aspect employed in the program offered is the employment of a static variable idCounter. The static variable idCounter is used to keep a count of the users registered till now and then assigns a distinct userID to each new user registering. Every time a new RegisteredUser object is created, the constructor method increments the idCounter by one and assigns its current value as the ID of the new user. This approach is highly effective because it ensures that every ID is distinct without employing any complicated logic.

RegisteredUser class also has two constructors: a default one and a non-default one. The default one sets the fields to reasonable default values for password and email, and a non-default one allows you to enter email and password directly. Both constructors are added as a nice gesture because then the program becomes convenient for the programmer. Default constructors are useful when testing is first done, whereas non-default constructors are easier to use in actual applications.

The program also contains toString() method inside the RegisteredUser class. The toString() method is defined to provide a well structured way to print out all user details. It's specifically useful for quickly confirming user registration or checking details while testing the program.

The Main class is the main part of the program, it handles all user interactions through menus and choices. It separates the user interface logic from the data handling logic, which

is another fundamental principle of Object Oriented Programming (separating concerns). The main method initializes an empty list of RegisteredUser objects to keep track of users. It also handles the menu options, and also calls methods like validateEmail() and validatePassword() to validate the input of the user and ensures the integrity of user data.

Placing the validation methods such as validateEmail() and validatePassword() inside the Main class as static methods works well here because these methods don't require any specific instance data. However, from an object oriented programming standpoint, these could also be moved into a separate "Validator" class to enhance the readability and modularity of the code. Doing this would make the program structure even cleaner and follow object oriented programming principles more efficiently.

Overall, this Java program demonstrates a practical understanding of object oriented programming design. The class structures make logical sense, data encapsulation is implemented appropriately, and responsibilities are clearly divided among classes and methods. While there is room for further refinementsuch as introducing separate utility classes for validation, the current design successfully delivers a clean, understandable, and maintainable solution.

Section C: Program Testing

Testing is a very important part of making sure that a program works the way it's supposed to. For this Java-based user registration system, I've created a testing plan that covers the most common actions a user might take, as well as edge cases that might cause issues. These tests are based on the expected behavior outlined in the assignment scenario and focus on input validation, menu navigation, account handling, and error conditions.

The test cases below are designed to make sure that each part of the program works correctly. I've also added my reasoning for why each test is useful and what it's supposed to prove.

Program Test Plan

Test Case	Input	Expected Output	Reason
Register with valid inputs	Email: user@test.com, Password: Password123	"User registered successfully", user ID is shown	Tests standard registration flow
Register with invalid email	Email: usertest.com, Password: Password123	"Invalid email. Try again."	Tests email validation logic
Register with weak password	Email: user@test.com, Password: pass	"Invalid password. Try again."	Ensures password strength rules are applied
Login with correct login details	Email: user@test.com, Password: Password123	"Login successful!", access to user menu	Tests successful login
Login with incorrect password	Email: user@test.com, Password: wrongpass (x3 attempts)	"Login failed." (after 3 tries), return to main menu	Tests login limit feature and retry handling
Change email after login	New Email: newuser@test.com	"Email updated successfully!"	Tests that email is updated properly
Change password after login	New Password: Newpass123	"Password updated successfully!"	Verifies password change feature
Print user details	-	Displays current user ID, email, and password	Confirms toString() method output
Delete account with confirmation	Input: yes	"Account deleted successfully!"	Tests account deletion and removal from list

Test Case	Input	Expected Output	Reason
Delete account and cancel	Input: no	Account not deleted, user returns to menu	Tests user cancellation handling
Multiple user registrations	Register 3 users	User IDs assigned: 1, 2, 3	Verifies user ID increment logic
Logout from user menu	Choose option 5	Return to main menu	Checks logout functionality
Invalid option from menu	Enter: 7	"Invalid option. Please try again."	Tests robustness against incorrect input

Why these test matter?

Each of these tests helps to confirm a different part of the program. Some of them are very straightforward, for example, checking if registration or login works. Others are more focused on edge cases, like entering an invalid email or trying to log in too many times with the wrong password.

It's also important to test things like changing email and password, deleting an account, and how the system behaves when someone tries to enter an invalid menu option. These situations are all realistic and can happen in real usage.

Testing multiple user registrations also confirms that the userID system works as intended and that each user gets a unique and incrementing ID. This is a key part of the scenario, so it has to be tested carefully.

Overall, this testing plan helps ensure that the system is not only working, but also reliable and user-friendly under different conditions. By covering both normal and unusual inputs, it gives a complete picture of how well the program performs.

Section D: Meeting the FIT1051 Coding Standards

After reviewing the Java code provided for this assignment, I found that it follows most of the FIT1051 coding standards quite well. These standards are important because they help make the code more readable, organized, and easier to maintain—not just for the original author, but also for others who may work with the code in the future. Below is a breakdown of how the current program aligns with these standards across the main areas.

1. Naming

The naming of variables, methods, and classes is mostly done correctly. For example, the class RegisteredUser uses a singular noun with an uppercase first letter, which matches the standard for class names. Variables such as userID, email, and password follow camel case conventions, and method names like validateEmail() and setPassword() are clear and use action verbs. There are no abbreviations or confusing names, which means the code is easy to understand even without excessive comments.

2. Operators and Expressions

Arithmetic, logical, and assignment operators are all spaced correctly, with a single space before and after the operator. This improves readability and avoids cluttered-looking expressions. For example, expressions like if (user.getEmail().equals(email)) are consistently formatted and easy to read.

3. Variables

The code respects the guideline that fields should only be initialized in constructors, and all local variables are declared and initialized properly when used. Public fields are avoided, which is good, and all variable names are in camel case with no underscores (except for the final static idCounter variable, which is acceptable as it is logically tied to the concept of a constant). However, the constant itself (idCounter) could be renamed in all caps as ID_COUNTER to align with the final variable naming convention.

4. Methods

The method names are descriptive and use camel casing starting with a lowercase letter. Mutators and accessors follow the naming convention (getSomething and setSomething). There's one small area for improvement: methods could be sorted alphabetically within the class to fully meet the standard, although this isn't always strictly necessary for small projects like this. The use of method comments could also be improved, as most methods currently do not have documentation above them explaining their purpose.

5. Classes

The structure of the RegisteredUser class is well-organized. It contains both a default and a non-default constructor, accessors and mutators for all fields, and a toString() method to display object details. This satisfies the full class template requirement. Additionally, the class follows the correct order: fields are declared first, followed by constructors, and then methods. The class is also kept in a single file, which matches the guideline of one class per file.

6. Brackets and Indentation

Indentation is consistent throughout the program. Each block is properly indented using four spaces, and the placement of opening and closing curly braces {} is correct—they appear on their own lines, not on the same line as headers. This makes the structure of the code visually clear.

7. Code Quality

All access modifiers like private and public are properly used and explicitly stated. There are no missing modifiers. Also, the code lines are mostly short and stay under 80 characters in length. In the few places where lines could potentially get too long, they could be split across multiple lines for better readability, but this is not a major issue.

8. Documentation

This is the main area where improvements could be made. The classes and methods currently lack descriptive comments. For example, adding a comment at the top of the RegisteredUser class to describe its purpose, the author's name, and version number would make the code more professional and meet the documentation standards. Also, short comments above each method would help explain what each function does, especially for someone new reading the code. At the same time, the code avoids excessive inline comments, which is a good thing.

Conclusion

To sum it up, the code largely follows the FIT1051 coding guidelines and is well-written in terms of structure, readability, and design. The only noticeable areas that could be improved are adding more formal documentation (like method comments and class headers) and alphabetizing methods for better organization. Other than that, the code meets the standard and shows a good understanding of clean Java programming practices.